

Keycloak ↔ Django Sync Architecture

Your application has **two separate user databases** that must stay in sync:

Database	Where it lives	What it stores	Purpose
Keycloak	Railway PostgreSQL (or local)	Username, password, email, custom attributes (role , ministry_schema), enabled status	Identity — "Who are you?" Password verification, SSO login
Django	Railway PostgreSQL (or local) <code>public.authentication_customuser</code>	Username, role , ministry_schema , is_active , is_locked , keycloak_id	Authorization — "What can you do?" Controls dashboard access, permissions

Every logged-in user exists in BOTH databases. They must be kept consistent — if you deactivate a user in Django but not in Keycloak, they can still log in via SSO (Keycloak accepts the password) but Django blocks them.

Which Files Do What

1. `authentication/oidc_backend.py` — The Bridge (runs on EVERY SSO login)

This is the most important file. Every time a user logs in via Keycloak SSO, mozilla-django-oidc calls these methods in order:

`filter_users_by_claims()` (line 12)

Searches for the Django user matching the Keycloak token.

If user exists in Django:

- Syncs **is_active** status **FROM Keycloak** (line 43–56) — if someone disabled the user in Keycloak, Django automatically disables them too
- Sets session flags for locked/deactivated accounts

If user NOT in Django:

- Returns empty queryset → mozilla-django-oidc calls `create_user()` next

`create_user()` (line 93) — "PendingAccess" flow

When a user exists in Keycloak but NOT in Django:

1. Reads **username**, **email**, **ministry_schema** from the Keycloak token (lines 95–99)
2. Creates a **PendingAccess** record (line 115) with those details

- Returns **None** → login is **denied** with "pending approval" message
- Super Admin sees the request in Django admin, approves, assigns role

update_user() (line 130) — Auto-sync role on every login

Runs on every successful login:

- Reads **role** and **ministry_schema** from Keycloak token claims (lines 133–134)
- If they differ from Django's stored values, updates Django (lines 141–148)
- This is why setting custom attributes in Keycloak is essential

get_user() (line 78)

Final gate: blocks users who are **is_locked=True** (brute-force lockout).

2. authentication/keycloak_admin.py — The Admin API Client

Talks to Keycloak's REST Admin API using admin credentials

(**KEYCLOAK_ADMIN_USERNAME/KEYCLOAK_ADMIN_PASSWORD**).

Method	Line	When it's called	What it does
update_user()	162	User edit, activate/deactivate	Syncs Django changes TO Keycloak
create_user()	56	Creating new user from Django admin	Creates user in Keycloak with attributes
reset_password()	277	Password reset	Sets new password in Keycloak
get_user()	187	Login sync	Fetches user status FROM Keycloak
get_all_users()	207	User list page	Syncs all users' is_active statuses
ensure_custom_attributes_defined()	222	setup_demo_data, sync_keycloak_attributes	Declares role/ministry_schema in realm's user profile (needed for Keycloak 26+)

3. authentication/user_views.py — Manual Sync Actions

These views let admins manage users. Each one syncs changes to BOTH databases:

View	Line	Direction	What syncs
user_toggle_active_view()	361	Django → Keycloak	is_active status (activate/deactivate)

View	Line	Direction	What syncs
user_edit_view()	208	Django → Keycloak	email, first_name, last_name, role, ministry_schema
user_reset_password_view()	469	Django → Keycloak	Password
user_create_view()	71	Keycloak first, then Django	Full user (with rollback if Django fails)
user_list_view()	11	Keycloak → Django (batch)	is_active status for all users on page load
_sync_user_from_keycloak()	583	Keycloak → Django (single)	is_active for one specific user

4. authentication/models.py — Django Models

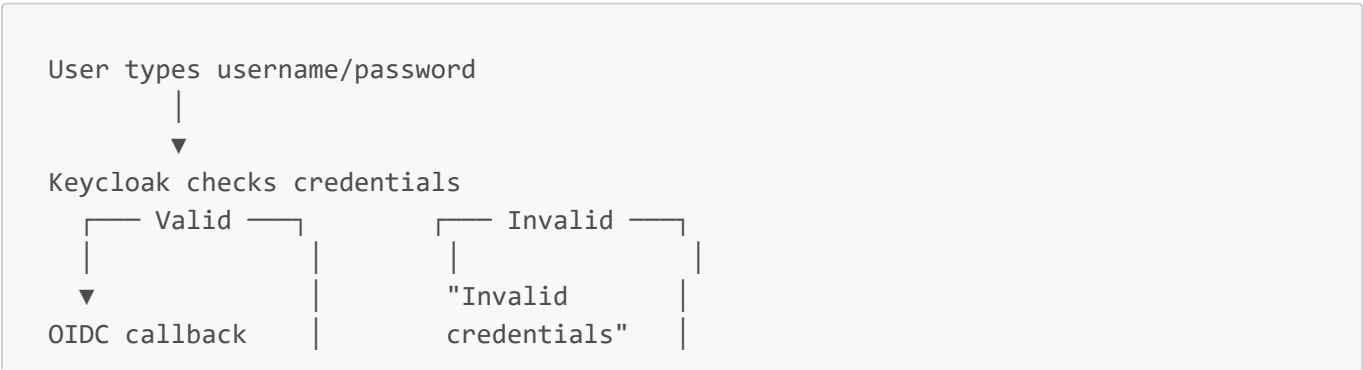
Model	Line	Purpose
CustomUser	11	Django's user with keycloak_id link, role, ministry_schema, is_locked
PendingAccess	69	Stores login attempts from Keycloak users who don't exist in Django yet

5. Management Commands

Command	File	Purpose
setup_demo_data	setup_demo_data.py	Creates 6 demo users in BOTH Django and Keycloak (with --sync-keycloak flag). Also seeds assets, categories, org units.
sync_keycloak_attributes	sync_keycloak_attributes.py	One-time fix: pushes role and ministry_schema attributes to existing Keycloak users who were created manually without them.

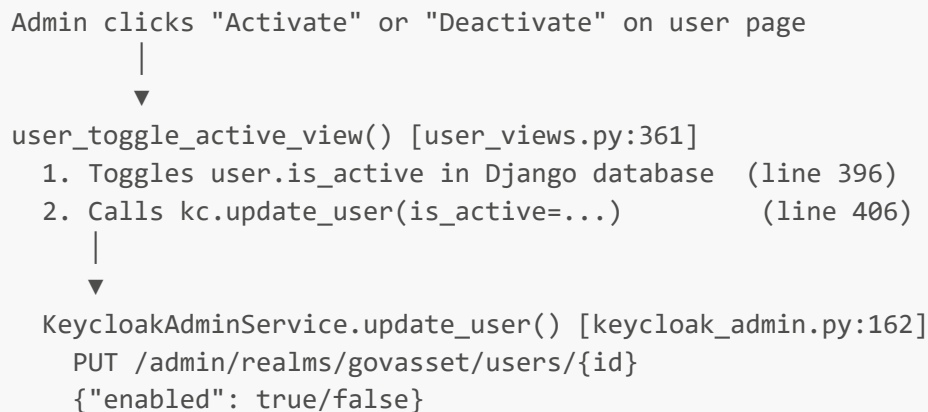
Complete Data Flows

Login Flow (SSO)

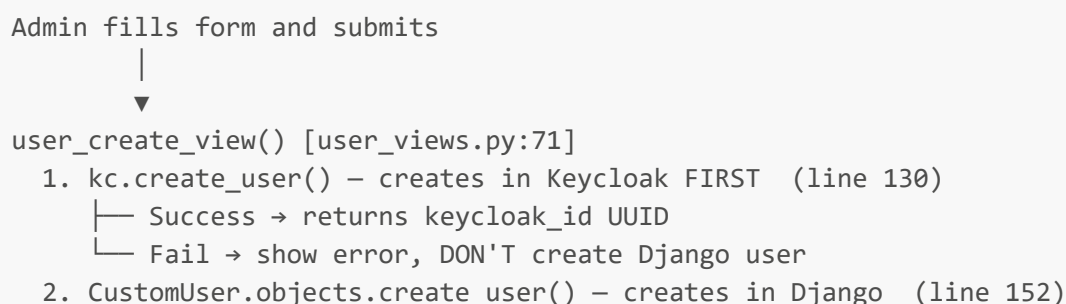




Activate/Deactivate Flow



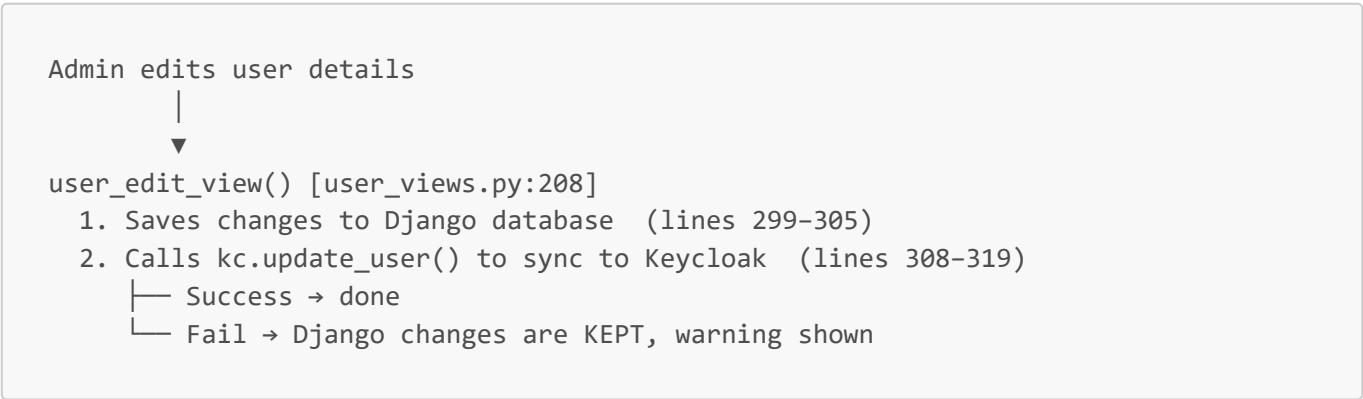
Create User Flow (from Django admin)



└─ Success → done

└─ Fail → kc.delete_user() ROLLBACK Keycloak (line 171)

Edit User Flow



The Custom Attributes: **role** and **ministry_schema**

These are stored as **Keycloak custom attributes** on each user. They are **not** standard Keycloak fields — they are set manually.

Why they exist

Django needs two pieces of information that Keycloak doesn't know about by default:

- **role** — What permission level (SUPER_ADMIN, MINISTRY_ADMIN, etc.)
- **ministry_schema** — Which ministry's data they can see (moh_schema, mof_schema)

These are stored in Keycloak's custom attributes so that on every login, **update_user()** in **oidc_backend.py** (line 130) can read them from the token claims and sync them to Django automatically.

How they are set

Method	How attributes get set
setup_demo_data --sync-keycloak	Automatically sets both attributes on user creation (line 292–293)
sync_keycloak_attributes	One-time fix: pushes attributes to existing users
Keycloak admin console → User → Attributes tab	Manual — only works in older Keycloak versions that show the tab
Keycloak admin console → User → Attributes tab	After declaring in User Profile (Realm settings → User profile)

Why Keycloak 26+ needs special handling

In Keycloak 26+, custom attributes are **not** accepted unless they are first **declared** in the realm's **User Profile** configuration. If you try to set an undeclared attribute via the API, Keycloak **silently drops it** (returns 204 but doesn't save).

The `ensure_custom_attributes_defined()` method in `keycloak_admin.py` (line 222) handles this by:

1. Fetching the current user profile config via `GET /admin/realms/{realm}/users/profile`
2. Checking if `role` and `ministry_schema` are already listed
3. If missing, adding them via `PUT /admin/realms/{realm}/users/profile`

This is called automatically by both `setup_demo_data` and `sync_keycloak_attributes` before any user operations.

The `sync_keycloak_attributes` Command

When to use: After creating users manually in the Keycloak admin console (or after deploying to a new Keycloak instance where users were imported/created without attributes).

What it does:

For each of 6 demo users:

1. Look up user by username in Keycloak (`kc.get_user_id`)
2. If found → update their `role` and `ministry_schema` attributes (`kc.update_user`)
3. If not found → skip with warning

It does NOT touch Django. It only updates Keycloak user records.

Troubleshooting Sync Issues

Symptom	Likely cause	Fix
"Status updated in system but Keycloak sync failed: 401"	Wrong admin credentials for Keycloak API	Set <code>KEYCLOAK_ADMIN_USERNAME</code> and <code>KEYCLOAK_ADMIN_PASSWORD</code> env vars on the Django service
User can log in but gets PendingAccess	User exists in Keycloak but not Django	Approve the PendingAccess record in Django admin
User logs in but has no permissions	<code>role</code> or <code>ministry_schema</code> missing from Keycloak attributes	Run <code>sync_keycloak_attributes</code> or set them manually
User shows as deactivated in Django but can still log in	Keycloak's <code>enabled</code> is still true	Toggle from Django user management (syncs to Keycloak) or disable in Keycloak admin
502 errors when managing users	Keycloak running out of memory	Check <code>JAVA_OPTS_APPEND=-Xmx256m -Xms128m</code> on Railway Keycloak service